

总体刚度矩阵组装与桁架结构求解程序设计报告

一、公式推导

1.1 一维杆单元

设杆单元两端节点坐标分别为 x_1 和 x_2 ，长度为：

$$L = x_2 - x_1$$

单元刚度矩阵（局部坐标系，自由度顺序 $[u_1, u_2]$ ）：

$$\mathbf{k}^{(e)} = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

1.2 二维桁架单元

设节点坐标 (x_1, y_1) 和 (x_2, y_2) ，长度：

$$L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

方向余弦：

$$c = \frac{x_2 - x_1}{L}, \quad s = \frac{y_2 - y_1}{L}$$

全局坐标系下的单元刚度矩阵（自由度顺序 $[u_1, v_1, u_2, v_2]$ ）：

$$\mathbf{K}^{(e)} = \frac{EA}{L} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}$$

1.3 应变-位移矩阵与应力

轴向应变为：

$$\epsilon = \frac{1}{L} \begin{bmatrix} -c & -s & c & s \end{bmatrix} \mathbf{d}^{(e)}$$

应力:

$$\sigma = E\epsilon$$

轴力:

$$N = \sigma A = EA\epsilon$$

二、程序主要函数说明

程序采用 Python 编写，依赖 NumPy 进行矩阵运算。

2.1 element_ke(nsd, E, A, L, c, s)

- **功能:** 计算单元刚度矩阵。
- **输入:** 维度 nsd (1 或 2)，弹性模量 E，截面积 A，长度 L，方向余弦 c, s。
- **输出:** 单元刚度矩阵 Ke (2×2 或 4×4)。
- **错误处理:** 若 $L < 1e-12$ ，调用前会抛出异常。

2.2 Assembly._build_LM()

- **功能:** 根据单元连接关系 IEN 和每节点自由度数 ndof 生成对号矩阵 LM。
- **映射规则:**
 - 一维: [n1*ndof, n2*ndof]
 - 二维: [n1*ndof, n1*ndof+1, n2*ndof, n2*ndof+1]

2.3 Assembly.assemble()

- **功能:** 总体刚度矩阵直接组装。
- **算法:** 双重循环散射累加 $K[lm[i], lm[j]] += Ke[i, j]$ 。

2.4 Solver.solve_reduced()

- **功能：**缩减法处理位移边界条件，求解未知位移和约束反力。

- **原理：**

$$K_{FF} * d_F = F_F - K_{FE} * d_E$$

其中 K_{FF} 为自由度的刚度子矩阵， d_F 为未知位移， F_F 为外载荷， K_{FE} 为耦合刚度， d_E 为已知位移。

- **输出：**完整位移向量 d ，反力向量 $reactions$ 。

2.5 PostProcessor.compute_all_elements()

- **功能：**计算所有单元的长度、方向余弦、应力、轴力。

三、两个验证算例的计算结果

3.1 算例 1：一维两单元杆结构

输入参数

- 节点坐标：1(0)，2(1)，3(2)
- 单元 1：E=100，A=1，L=1；单元 2：E=200，A=1，L=1
- 边界条件： $d_1 = 0$
- 节点载荷： $F_3 = 10$

程序输出

总体刚度矩阵 K （未处理边界）：

$\begin{bmatrix} 100. & -100. & 0. \end{bmatrix}$

$\begin{bmatrix} -100. & 300. & -200. \end{bmatrix}$

$\begin{bmatrix} 0. & -200. & 200. \end{bmatrix}$

节点位移： 节点 1: $u = 0.000000$ 节点 2: $u = 0.100000$ 节点 3: $u = 0.150000$

约束反力： 自由度 1 (节点 1): $R = -10.000000$

单元 1: 应力 = 10.000000 , 轴力 = 10.000000 单元 2: 应力 = 10.000000 , 轴力 = 10.000000

验证结果对比

项目	理论值	程序输出	是否匹配
节点 2 位移	0.1	0.100000	✓
节点 3 位移	0.15	0.150000	✓
节点 1 反力	-10	-10.000000	✓
刚度矩阵	见上	完全一致	✓

3.2 算例 2：二维两杆桁架结构

输入参数

- 节点坐标：1 (1, 0), 2 (0, 0), 3 (1, 1)
- $E=1$, $A=1$ (两个单元)
- 节点 1、2 完全固定
- 节点 3 受水平力 $F_x = 10$

程序输出 总体刚度矩阵 K (部分):

$[[0. 0. 0. 0. 0. 0.]]$

$[0. 1. 0. 0. 0. -1.]$

$[0. 0. 0.3536 0.3536 -0.3536 -0.3536]$

[0. 0. 0.3536 0.3536 -0.3536 -0.3536]

[0. 0. -0.3536 -0.3536 0.3536 0.3536]

[0. -1. -0.3536 -0.3536 0.3536 1.3536]]

节点位移： 节点 3: $u = 38.284271$, $v = -10.000000$

约束反力： 自由度 1 (节点 1, u): $R = 0.000000$ 自由度 2 (节点 1, v): $R = 10.000000$ 自由度 3 (节点 2, u): $R = -10.000000$ 自由度 4 (节点 2, v): $R = -10.000000$

单元 1: 应力 = -10.000000 , 轴力 = -10.000000 单元 2: 应力 = 14.142136 , 轴力 = 14.142136

验证结果对比

项目	理论值	程序输出	是否匹配
节点 3 u	38.284271	38.284271	✓
节点 3 v	-10.000000	-10.000000	✓
单元 1 应力	-10.000000	-10.000000	✓
单元 2 应力	14.142136	14.142136	✓
刚度矩阵对称	对称	True	✓

四、总体刚度矩阵性质验证

4.1 对称性验证

程序输出显示 K 满足 `np.allclose(K, K.T)`, 两个算例均输出 `True`。

4.2 奇异性验证（行列式分析）

状态	算例 1	算例 2
处理边界前	$\det(K) \approx 0$	$\det(K) \approx 0$
处理后（缩减矩阵）	$\det(K_{FF}) = 2 \times 10^4$	$\det(K_{FF}) = 3.54 \times 10^{-1}$

说明原始刚度矩阵奇异（存在刚体位移），施加足够约束后变为非奇异。

4.3 对角元非负性

所有对角元 $K_{ii} \geq 0$ ，满足物理要求。

4.4 稀疏性

- 算例 1（3 自由度）：非零元素比例 77.78%
- 算例 2（6 自由度）：非零元素比例 52.78%

对于更大规模问题，稀疏性会更加显著。

4.5 物理意义验证（总体刚度矩阵第 j 列）

选取总体自由度 $j = 5$ （即算例 2 中节点 3 的 u 方向），令 $d_5 = 1$ ，其余位移为 0，计算节点力：

$$F = K \cdot [0, 0, 0, 0, 1, 0]^T$$

所得结果正好等于 K 的第 5 列。验证了 K_{ij} 的物理意义：第 j 个自由度产生单位位移时，在第 i 个自由度上需要施加的节点力。

五、附加题：罚函数法实现与对比

程序中已实现罚函数法（`Solver.solve_penalty`）。以算例 2 为例，取罚因子 $\alpha = 10^{12}$ ，结果对比如下：

方法	节点 3 u	节点 3 v	最大反力误差
----	--------	--------	--------

缩减法	38.284271	-10.000000	0
-----	-----------	------------	---

罚函数法	38.284271	-10.000000	$<10^{-6}$
------	-----------	------------	------------

两种方法结果高度一致，罚函数法实现简单，但需注意数值稳定性。

六、结论

本程序完整实现了杆单元（一维）和二维桁架单元的有限元分析流程，包括：

- 对号矩阵 LM 的自动生成
- 总体刚度矩阵的直接组装
- 缩减法（及罚函数法）处理位移边界条件
- 节点位移、约束反力、单元应力/轴力的计算与输出

通过两个经典算例验证了程序的正确性。对总体刚度矩阵的对称性、奇异性、对角元非负性、稀疏性以及第 j 列的物理意义进行了分析，结果与理论完全吻合。程序结构清晰，包含错误处理，满足课程作业全部要求。